Relatório executivo EasyShop

Visão geral

O EasyShop implementa cadastro e listagem de produtos em LiteDB e clientes em SQLite, com interface em português acessada pelo navegador e servidor local ASP.NET Core. A entrega técnica foi executada em 11/11/2026, sob supervisão do aluno Gabriel Tavares. O prazo acadêmico informado é 30/11/2026. O vídeo permanece sob responsabilidade do colaborador.

Decisões arquiteturais

O workspace estava vazio e não havia SDK .NET disponível no PATH. Foi instalado o SDK 10.0.404 em diretório do usuário, sem alteração do PATH global. A stack escolhida foi C#/.NET 10, ASP.NET Core, HTML/CSS/JavaScript, LiteDB 5.0.21 e Microsoft.Data.Sqlite 10.0.15. O usuário autorizou seguir a melhor opção na ausência de plataforma obrigatória.

A interface web local evita a instalação de workloads Android e mantém compatibilidade com as bibliotecas exigidas. Não foi gerado APK nem realizado teste em aparelho físico. Uma eventual exigência Android depende de confirmação e implementação adicional. Caixa, dívidas, frotas, viagens, dashboards, RBAC e sincronização estão não aplicáveis ao escopo EasyShop - pendentes de confirmação.

UI chama endpoints HTTP; serviços aplicam regras; repositórios fazem persistência. A injeção de dependências controla os tempos de vida. SQLite utiliza parâmetros SQL e restrição de unicidade; LiteDB armazena produtos como documentos com decimal, lista de categorias e dicionário de atributos. Mensagens ao usuário omitem detalhes internos de falhas. O servidor escuta somente no loopback.

Detalhamento dos módulos

Produtos: cadastro, listagem, preço positivo com duas casas decimais, categorias e atributos opcionais, validação de tamanho e consolidação de categorias repetidas. Produtos de mesmo nome são permitidos. Não há controle de quantidade de estoque.

Clientes: nome e sobrenome obrigatórios, e-mail validado e normalizado, cadastro e listagem. Restrição única no SQLite protege a integridade mesmo com concorrência. Não há envio de e-mail nem verificação de titularidade.

Matriz de rastreabilidade

Requisito	Implementação	Evidência	Status
Produtos LiteDB	ProductService e ProductRepository	Testes de reabertura; produtos.png	Implementado e validado
Clientes SQLite	CustomerService e CustomerRepository	Testes de normalização e persistência; clientes.png	Implementado e validado
Arquitetura em camadas	wwwroot, Services, Data, Models	Código e diagrama README	Implementado
Regras e duplicidade	Serviços e restrição UNIQUE	11 testes xUnit; email-duplicado.png	Validado

Requisito	Implementação	Evidência	Status
Interface e estados	HTML, CSS e app.js	ui-results.json e capturas reais	Validado em navegador automatizado
Build e análise	.NET build e format	build.txt; comandos registrados	Aprovado sem avisos
Empacotamento	Publish Windows x64 autocontido	artifacts/publish/win-x64	Gerado
Documentação	README, manual, relatório, roteiro	Arquivos em docs	Entregue
Repositório público	GitHub, após autorização explícita	Consultar docs/entrega.md	Ver link e estado final no checklist
Convite ao colaborador	GitHub	Usuário não informado	Pendente
Vídeo 3 a 5 minutos	Roteiro de 4 minutos	docs/video/roteiro.md	Roteiro entregue; gravação e link pendentes
Android e logística/finanças	Sem alvo ou módulos implementados	Decisão de escopo	Não aplicável ao escopo EasyShop - pendente de confirmação

Validação e evidências

Build Release aprovado sem avisos nem erros. Os 11 casos xUnit passaram: produtos inválidos, reabertura LiteDB, clientes inválidos, normalização, reabertura SQLite e duplicidade concorrente. O teste de interface Playwright executou cadastro real, listagem, preço zero, duplicidade, estados vazios e viewport de 390 pixels sem overflow horizontal, sem erros JavaScript. Essas capturas são reais, obtidas da aplicação com dados fictícios; não são mockups. A sessão de navegador foi automatizada, sem alegação de teste humano ou aparelho físico.

`dotnet format --verify-no-changes --no-restore` aprovado. A auditoria NuGet não reportou pacotes vulneráveis na consulta registrada. Isso representa o estado da fonte de avisos no momento da consulta e não garante ausência de toda vulnerabilidade possível. O registro de restart e a execução do pacote estão em evidencias/restart-results.json.

Infraestrutura e custos

Execução exclusivamente local, sem recursos de nuvem provisionados. Nenhum serviço pago foi contratado nesta execução. O hardware e a conexão já pertencem ao usuário; valores de aquisição, energia e internet estão a confirmar. Não há custo financeiro comprovado informado. Publicação do código utiliza repositório público GitHub autorizado pelo usuário.

Cronograma planejado e executado

Cronograma original de sprints: a confirmar, não fornecido. Prazo planejado informado: 30/11/2026. Em 11/11/2026 foram realizados auditoria do ambiente, instalação do SDK, implementação dos dois módulos, testes automatizados, capturas e documentação. Gravação, publicação do vídeo e confirmação de exigências acadêmicas permanecem pendentes. Não foram inventadas sprints ou horas de esforço.

Dificuldades e lições aprendidas

A ausência inicial do SDK foi resolvida por instalação no perfil do usuário. A primeira restauração detectou aviso de vulnerabilidade em SQLitePCLRaw 2.1.11; a atualização de Microsoft.Data.Sqlite trouxe a versão transitiva 2.1.13 e a consulta de vulnerabilidades deixou de apresentar alertas. O teste de interface exigiu ajuste do seletor do campo de e-mail para incluir a marca visual de obrigatório; houve também correção de codificação do script de teste após leitura pelo PowerShell. Esses ajustes foram seguidos de nova execução.

A separação das regras de negócio permitiu testes com bancos isolados, sem depender da UI. A unicidade deve ser garantida pelo banco além da validação de formulário. Evidências reproduzíveis e identificação clara de pendências evitam confundir protótipo demonstrável com entrega acadêmica final.

Limitações e próximos passos

Sem edição, exclusão, login, criptografia de banco, paginação, sincronização ou acesso em rede. Adequado à demonstração acadêmica local, não a serviço multiusuário público. Não há teste Android nem obrigação de plataforma confirmada. Para concluir a submissão, informar colaborador, gravar e publicar vídeo, verificar acesso aos links e validar com o professor se há outro enunciado aplicável.